

Documentation Technique

Application Production Adjointe

Laboratoire Argoat
Version du 25/06/2026

Application de gestion de production pour laboratoire de protheses dentaires.
Suivi en temps reel des cas a travers 5 secteurs de production, de la conception a la finition.

Stack technique : Next.js 16 (App Router) + Supabase (PostgreSQL) + Vercel

Repository : github.com/antoineangelini-eng/PRODUCTION-ADJOINTE

URL de production : Deploye sur Vercel (region Paris cdg1)

Sommaire

1. Vue d'ensemble et architecture
2. Structure du projet
3. Base de donnees (Supabase)
4. Les 5 secteurs de production
5. Flux de travail d'un cas
6. Authentification et securite
7. Systeme d'impression (Zebra)
8. Composants principaux
9. Variables d'environnement
10. Dependances et librairies
11. Guide de transfert de compte

Document genere le 25/06/2026

1. Vue d'ensemble et architecture

Production Adjointe est une application web de gestion de production pour le Laboratoire Argoat. Elle permet de suivre chaque cas (prothese dentaire) a travers les differents secteurs de fabrication, depuis la conception jusqu'a la finition et l'expedition.

Architecture globale

Couche	Technologie	Role
Frontend	Next.js 16 + React 19 + Tailwind CSS 4	Interface utilisateur, rendu serveur
Backend	Next.js Server Actions + API Routes	Logique metier, RPC Supabase
Base de donnees	Supabase (PostgreSQL)	Stockage, authentication, RPC
Hebergement	Vercel (region Paris cdg1)	Deploiement automatique via GitHub
Impression	Print Relay (Node.js sur LAN)	Envoi ZPL aux imprimantes Zebra
Repository	GitHub	Versioning, CI/CD avec Vercel

Flux de donnees

1. L'utilisateur interagit avec l'interface Next.js dans son navigateur.
2. Les actions declenchent des **Server Actions** qui appellent des **fonctions RPC Supabase**.
3. Les fonctions RPC executent la logique metier directement en PostgreSQL (creation, mise a jour, routage).
4. Les tableaux se rafraichissent automatiquement toutes les **30 secondes** via polling.
5. Pour l'impression, le serveur genere du code ZPL, le navigateur l'envoie au **Print Relay** local qui transmet a l'imprimante.

Pourquoi un Print Relay ? L'application tourne sur Vercel (cloud) et ne peut pas atteindre directement les imprimantes du reseau local. Le Print Relay est un petit serveur Node.js qui tourne sur une machine du LAN (port 3001) et fait le pont entre le cloud et les imprimantes.

2. Structure du projet

Dossier	Contenu	Description
app/	Pages + Actions	Routes Next.js App Router, layouts, server actions
app/app/	Pages secteurs	Design Metal, Design Resine, UT, UR, Finition, Admin
app/login/	Page login	Authentification email/mot de passe
components/	~60 fichiers	Composants React reutilisables
components/sheet/	~56 fichiers	Tableaux, cellules editables, modales, scanners
columns/	Definitions	Colonnes de tableaux et types partages

hooks/	1 hook	usePollingRefresh (refresh automatique 30s)
lib/	Utilitaires	Clients Supabase (4 variantes), zebra-print.ts
print-relay/	Serveur Node	Relais d'impression LAN vers imprimantes Zebra
supabase/	Migrations	12 fichiers de migration SQL (avril 2026)
public/	Assets	Images SVG statiques

Routes de l'application

URL	Page	Acces
/login	Connexion	Public
/app	Redirection auto vers le 1er secteur	Authentifie
/app/design-metal	Secteur Design Metal (+ /history)	DM ou Admin
/app/design-resine	Secteur Design Resine	DR ou Admin
/app/usinage-titane	Secteur Usinage Titane	UT ou Admin
/app/usinage-resine	Secteur Usinage Resine	UR ou Admin
/app/finition	Secteur Finition	FI ou Admin
/app/history	Historique global	Tous
/app/admin	Administration	Admin uniquement

3. Base de donnees (Supabase)

16 tables dans le schema **public**, structurees autour de la table centrale **cases**.

Tables principales

Table	Role	Colonnes cles
cases	Cas principal	id, case_number, nature_du_travail, date_expedition, is_physical
case_assignments	Affectation secteur	case_id, sector_code, status (active/in_progress/done/on_hold)
case_events	Journal d'audit	case_id, event_type, created_by, payload (JSONB)

Tables par secteur

Table	Secteur	Colonnes cles
sector_design_metal	Design Metal	design_chassis, dentall_case_number, reception_metal_date, teintes_associees
sector_design_resine	Design Resine	design_dents_resine, nb_blocs_de_dents, base_type, teintes_associees, complet
sector_usinage_titane	Usinage Titane	machine_ut, numero_calcul, nombre_brut, reception_metal (+_h/_b)
sector_usinage_resine	Usinage Resine	identite_machine, numero_disque, numero_lot_pmma, nb_blocs_override
sector_finition	Finition	validation, reception_metal_ok, reception_resine_ok (+timestamps)

Tables de support

Table	Role
profiles	Profils utilisateurs : secteurs assignes, display_name, email
user_printers	Association utilisateur <-> adresse IP imprimante
user_display_names	Vue pour afficher les noms (jointure auth.users)
sectors	Configuration des secteurs (noms, codes)
sector_routes	Routage : quelle nature va vers quels secteurs
working_days_config	Jours ouvres par nature pour calcul expedition
announcements	Annonces cibles par secteur
announcement_dismissals	Suivi des annonces lues par utilisateur
feedback	Retours utilisateurs (titre, description, priorite)
case_queue	File d'attente temporaire (vide en general)

Fonctions RPC Supabase (18 fonctions)

Categorie	Fonctions
Creation	rpc_create_case_from_design_metal, rpc_create_case_from_design_resine
Mise a jour	rpc_update_design_metal, _design_resine, _usage_titane, _usage_resine, _finition
Completion	rpc_complete_design_metal, _design_resine, _usage_titane, _usage_resine
Validation	rpc_validate_finition_batch
Expedition	rpc_update_case_expedition
Scan	rpc_scan_case
Physique	rpc_mark_case_physical, rpc_toggle_case_physical
Admin	rpc_admin_reset_sectors, rpc_reopen_case

4. Les 5 secteurs de production

Design Metal (DM)

Point d'entree pour les cas necessitant du metal (Chassis Argoat, Chassis Dent All, Definitif Resine).

Creation du cas, saisie du design chassis, envoi Dent All, gestion des teintes. A la completion, le cas est route vers les secteurs suivants (UT, DR, Finition selon la nature).

Design Resine (DR)

Point d'entree pour les cas resine (Provisoire, Deflex, Complet) et etape intermediaire pour les cas venant de DM.

Design des dents resine, gestion des blocs, type de base, teintes. A la completion, le cas part vers Usinage Resine et/ou Finition.

Usinage Titane (UT)

Usinage des chassis metalliques.

Saisie machine, numero de calcul, nombre de bruts. Supporte le mode H/B (Haut/Bas) pour separer les donnees. Impression d'etiquettes avec machine/calcul/brut. Gestion par lots (scan + saisie groupée).

Usinage Resine (UR)

Usinage des elements en resine.

Saisie machine (PM + IMES), disque, lot PMMA, teintes, blocs. Impression d'etiquettes detaillees (DENTS + BASE). Machines reparties en 2 groupes : PM (gauche) et IMES (droite) dans le dropdown.

Finition (FI)

Etape finale avant expedition.

Scanner les cas pour confirmer la reception metal et/ou resine. Validation automatique quand toutes les receptions sont faites. Supporte le multi-nature (meme numero de cas, natures differentes). Impression d'etiquettes a la validation.

5. Flux de travail d'un cas

Routage par nature du travail

Nature	Entree	Flux
Chassis Argoat	Design Metal	DM -> UT + DR + Finition, puis DR -> UR
Chassis Dent All	Design Metal	DM -> DR + Finition (pas de UT), puis DR -> UR
Definitif Resine	Design Metal	DM -> DR + Finition, puis DR -> UR
Provisoire Resine	Design Resine	DR -> UR + Finition
Deflex	Design Resine	DR -> UR + Finition
Complet	Design Resine	DR -> UR + Finition

Cycle de vie d'un cas

- 1. Creation** : Saisie du numero de cas (ou scan code-barres) dans DM ou DR. Calcul automatique de la date d'expedition basee sur les jours ouvres configures par nature.
- 2. Traitement** : Le cas apparait dans le tableau du secteur avec le statut **active**. L'utilisateur remplit les informations (machine, teintes, etc.) et les cellules se sauvegardent automatiquement au blur.
- 3. Completion** : Selection des cas termines + clic sur le bouton de completion. La fonction RPC complete le secteur et active automatiquement les secteurs suivants selon le routage.
- 4. Finition** : Scan des cas pour confirmer la reception des elements (metal et/ou resine). Validation automatique quand tout est reçu. Impression de l'etiquette finale.
- 5. Historique** : Les cas valides en Finition passent dans l'historique de chaque secteur.

Statuts d'un cas dans un secteur

Statut	Description	Visible dans
active	Cas en attente de traitement	Tableau actif du secteur
in_progress	Cas en cours de traitement	Tableau actif du secteur
on_hold	Cas mis en attente (avec raison)	Tableau actif (surbrillance)
done	Secteur termine, cas route ou archive	Historique du secteur

Cas speciaux

Mode H/B (Usinage Titane) : Certains cas necessitent des donnees separees Haut/Bas pour machine, calcul et brut. Un toggle active ce mode et dedouble les champs.

Multi-nature (Finition) : Un meme numero de cas peut avoir plusieurs natures (ex: 133070 avec 'Chassis Dent All' et 'Complet'). Le scanner gere ce cas en excluant les IDs deja scannes.

Auto-validation (Finition) : Si un cas n'a pas besoin de la reception en cours (ex: pas besoin de metal) mais que toutes ses autres receptions sont faites, il est auto-valide.

Dents du commerce / Pas de dents : Si le type de dents est l'un de ces deux, le Design Resine est auto-complete (skippe) et la Finition ne requiert pas la reception resine.

6. Authentification et securite

L'authentification est geree par **Supabase Auth** (email + mot de passe). Le middleware Next.js rafraichit le token de session a chaque requete. Le layout principal (`/app/app/layout.tsx`) verifie l'authentification et redirige vers /login si necessaire.

Niveaux d'accès

Role	Acces	Capacites
Utilisateur	Ses secteurs assignes uniquement	Voir/editer les cas, completer, imprimer
Multi-secteurs	Plusieurs secteurs	Meme que utilisateur sur chaque secteur
Admin	Tous les secteurs + admin	Gestion utilisateurs, suppression, reouverture de cas, vue globale

Clients Supabase (4 variantes)

Client	Utilisation	Cle
Server (SSR)	Server Actions, rendu serveur	Anon key + cookies utilisateur (RLS actif)
Client (Browser)	Composants client React	Anon key (RLS actif)
Admin	Operations privilegiees	Service Role Key (bypass RLS)
Middleware	Refresh de session	Anon key + cookies

Compte protege : L'adresse `antoine.angelini@labo-argoat.fr` est codee en dur comme non-supprimable et ne peut pas perdre le role admin. Cela garantit qu'il y a toujours un administrateur dans le systeme.

7. Systeme d'impression (Zebra)

Architecture d'impression en 3 couches

- 1. Server Action** (Vercel) : Genere le code ZPL a partir des donnees du cas. Recupere l'IP de l'imprimante assignee a l'utilisateur.
- 2. Navigateur** (sur le LAN) : Recoit le ZPL et l'envoie au Print Relay via HTTP POST.
- 3. Print Relay** (serveur Node.js local, port 3001) : Recoit le ZPL et le transmet en TCP brut sur le port 9100 de l'imprimante Zebra. 3 tentatives avec 1500ms de delai.

Formats d'etiquettes

Format	Utilisation	Contenu
--------	-------------	---------

buildZPL (detail)	Usinage Resine	Teinte, blocs, machine, disque, modele + section BASE
buildSimpleZPL (compact)	Usinage Titane	Code-barres, nature, machine/calcul/brut (bandeau noir), dates, modele, qte
buildSimpleZPL (standard)	Finition	Code-barres, nature, dates, modele (sans section usinage)
EMAX (client-side)	Palettes EMAX	Genere cote client

Demarrage du Print Relay

Sur la machine du LAN qui fait tourner le relay :

```
cd print-relay  
node server.js
```

Le serveur démarre sur le port 3001. L'URL du relay est configurée dans la variable d'environnement NEXT_PUBLIC_PRINT_RELAY_URL (default : <http://192.168.1.30:3001>).

8. Composants principaux

~60 composants dans `/components/sheet/`, organisés par pattern :

Type	Exemples	Role
PageClient (6)	DesignMetalPageClient, AdminPageClient...	Shell de page : etat, polling, orchestration
Table (5)	DesignMetalTable, UsinageTitaneTable...	Grille de donnees avec edition inline, selection, actions batch
BoolCell (4)	BoolCell, BoolCellModele...	Cases a cocher editables (sauvegarde au clic)
TextCell (4)	TextCell, SelectCell...	Champs texte/select editables (sauvegarde au blur)
DateCell (3)	DateCell, ScrollCalendar...	Selecteur de date avec calendrier francais
CreateBar (2)	CreateBarDM, CreateBarDR	Barre de creation avec scan code-barres (AZERTY)
LotPanel (4)	LotPanelTitane, LotPanelUR...	Saisie groupée : scan multiples cas + champs communs
BatchValidate (2)	BatchValidate, BatchComplete	Completion/validation en lot avec aperçu
Scanner (1)	FinitionScanner	Scan + verification reception + auto-validation
Modales (3)	CaseDetailModal, DeleteConfirm...	Detail cas, suppression, mise en attente
Bannières (3)	RealtimeBanner, IncomingCases...	Notifications temps reel, nouveaux cas
Admin (4)	UsersManager, PrintersManager...	Gestion utilisateurs, imprimantes, jours ouverts

Mecanisme de rafraichissement

Le hook **usePollingRefresh** gere le rafraichissement automatique toutes les **30 secondes**. Il detecte si l'utilisateur est en train d'editer (selection active, cellule en edition) et dans ce cas differe le refresh jusqu'a ce qu'il ait fini. Les nouveaux cas apparaissent sous forme de toasts et de bannières sans perturber la vue en cours.

9. Variables d'environnement

Variable	Obligatoire	Description
NEXT_PUBLIC_SUPABASE_URL	Oui	URL du projet Supabase (https://xxx.supabase.co)
NEXT_PUBLIC_SUPABASE_ANON_KEY	Oui	Cle anonyme Supabase (cote client)
SUPABASE_SERVICE_ROLE_KEY	Oui	Cle service role (cote serveur uniquement, bypass RLS)
NEXT_PUBLIC_PRINT_RELAY_URL	Non	URL du relais d'impression (default: http://192.168.1.30:3001)

Important : Les variables prefixees **NEXT_PUBLIC_** sont exposees cote client (navigateur). La **SUPABASE_SERVICE_ROLE_KEY** ne doit JAMAIS etre exposee cote client — elle bypass toutes les regles de securite RLS. Sur Vercel, ces variables sont configurees dans Project Settings > Environment Variables.

10. Dependances et librairies

Le projet est tres leger avec seulement **5 dependances de production** :

Package	Version	Role
next	16.1.6	Framework React avec rendu serveur
react + react-dom	19.2.3	Librairie UI
@supabase/supabase-js	^2.97.0	Client Supabase (requetes, auth)
@supabase/ssr	^0.8.0	Supabase pour le rendu serveur (cookies, middleware)
tailwindcss	^4	Framework CSS utilitaire (devDep)
typescript	^5	Typage statique (devDep)

11. Guide de transfert de compte

Pour transférer le projet vers un nouveau compte (nouvelle adresse mail), voici les étapes pour chaque service :

GitHub

1. Créer le nouveau compte GitHub avec la nouvelle adresse mail.
2. Sur l'ancien compte, aller dans le repo PRODUCTION-ADJOINTE > Settings > Danger Zone > Transfer.
3. Saisir le nom du nouveau compte comme destinataire.
4. Le nouveau compte accepte le transfert.
5. Toute l'historique Git, les branches et les issues sont conservées.

Vercel

1. Créer le nouveau compte Vercel et passer en Pro (20 \$/mois).
2. Connecter le nouveau compte Vercel au nouveau compte GitHub.
3. Sur l'ancien compte Vercel : Project Settings > General > Transfer Project.
4. Sélectionner le nouveau compte comme destinataire.
5. Les variables d'environnement, le domaine et la config suivent automatiquement.
6. L'ancien compte peut ensuite être downgraded en gratuit ou supprimé.

Supabase

1. Créer le nouveau compte Supabase avec la nouvelle adresse mail.
2. Sur l'ancien compte : Organization Settings > Members > Inviter le nouveau compte en tant que Owner.
3. Le nouveau compte accepte l'invitation et devient Owner de l'organisation.
4. Retirer l'ancien compte de l'organisation.
5. Le nouveau compte passe en Pro (25 \$/mois) si nécessaire.
6. Alternative : créer un nouveau projet et migrer les données via export SQL/CSV.

Attention : Lors du transfert, il faut mettre à jour les variables d'environnement sur Vercel si le projet Supabase change (nouvelles clés, nouvelle URL). Vérifier que l'adresse admin protégée dans le code est mise à jour si nécessaire.

Connexion de nouveaux projets

Pour connecter un nouveau projet à la même infrastructure :

Même base Supabase : Le nouveau projet peut utiliser la même base de données Supabase en partageant les mêmes clés (URL + anon key + service role key). Les tables sont partagées. Utile pour un projet complémentaire (ex: interface patient, tableau de bord direction).

Projet indépendant : Créer un nouveau projet Supabase avec sa propre base. Sur Vercel, déployer un nouveau projet connecté au nouveau repo GitHub. Chaque projet a ses propres variables d'environnement et sa propre base.

